

Архитектура и стратегия реализации суверенного VPN-сервиса на базе Marzban и Telegram Mini App в условиях технологического ландшафта 2026 года

Развитие глобальной сетевой инфраструктуры к 2026 году привело к необходимости создания качественно новых инструментов обеспечения приватности, способных функционировать в условиях продвинутого глубокого анализа трафика (DPI) и стратегий «суверенного интернета». Проект VPN-сервиса, интегрированного в экосистему Telegram через механизмы Mini App и управляемого мощным ядром Xray с оболочкой Marzban, представляет собой высокотехнологичный ответ на вызовы цифровой суверенности. Данный отчет содержит исчерпывающее исследование архитектурных решений, протоколов обфускации трафика, финансовых интеграций и методологии развертывания системы, ориентированной на обеспечение максимальной анонимности и отказоустойчивости.

Технологический фундамент и выбор протоколов обфускации

Основой современного VPN-решения является выбор протоколов, способных имитировать легитимный трафик, не вызывая подозрений у систем мониторинга. В рамках проекта выбрана связка VLESS и Reality, которая на текущий момент является эталоном в области противодействия активному зондированию.¹ В отличие от традиционных методов инкапсуляции трафика, протокол VLESS лишен избыточного шифрования на уровне самого протокола, делегируя эту задачу транспортному уровню, что снижает вычислительную нагрузку и вероятность обнаружения по статистическим характеристикам пакетов.²

Технология Reality в сочетании с VLESS позволяет полностью отказаться от использования собственных SSL/TLS сертификатов, которые часто становятся точкой деанонимизации серверов. Вместо этого Reality заимствует («крадет») параметры рукопожатия у существующих популярных веб-ресурсов, поддерживающих TLS 1.3, таких как Google, Microsoft или Yahoo.³ Для систем DPI такой трафик выглядит как стандартное обращение к доверенному серверу, что делает блокировку практически невозможной без нарушения работы значительной части глобальной сети.

Сравнительный анализ характеристик Reality и традиционных

методов TLS

Характеристика	VLESS + Reality	VMess + TLS	Trojan
Необходимость сертификата	Нет (используется чужой)	Да (нужен свой домен)	Да (нужен свой домен)
Защита от зондирования	Максимальная (технология Cloaking)	Средняя (Fingerprinting)	Высокая (имитация HTTPS)
Вычислительные затраты	Минимальные	Высокие	Средние
Сложность настройки	Высокая	Средняя	Низкая
Устойчивость к DPI 2026	Исключительная	Уязвима к активному поиску	Уязвима к анализу поведения

Реализация данной технологии требует точной настройки параметров в ядре Xray, включая генерацию пар ключей x25519 и выбор целевого домена (Dest), который должен находиться вне юрисдикции CDN и поддерживать современные алгоритмы шифрования.¹

Управление инфраструктурой через Marzban API

Marzban выступает в роли интеллектуального оркестратора, написанного на Python и React, который предоставляет REST API для управления учетными записями пользователей, мониторинга потребления трафика и управления нодами.⁵ В контексте разработки Telegram-бота использование Marzban позволяет абстрагироваться от низкоуровневой конфигурации JSON-файлов Xray, переводя взаимодействие в плоскость высокоуровневых запросов.

Ключевые возможности Marzban для VPN-сервиса

Программный комплекс Marzban обеспечивает полную автоматизацию жизненного цикла подписки:

- Создание пользователей с динамически генерируемыми ссылками для VLESS.⁷
- Ограничение по объему данных (Data Limit) и времени действия (Expiration Date).⁵
- Поддержка мульти-протокольных инбаундов, позволяющая одному пользователю подключаться через разные порты или протоколы в рамках одного аккаунта.⁵
- Возможность распределения нагрузки через систему Marzban-node, что критически важно для масштабирования сервиса в разных географических регионах.⁸

Интеграция с Python-библиотеками, такими как aiomarzban, позволяет реализовывать асинхронные обработчики для Telegram-бота, обеспечивая мгновенную реакцию на действия пользователя даже при высокой нагрузке на систему.⁹

Архитектура Backend-составляющей на aiogram 3.x

Выбор aiogram 3.x в качестве основного фреймворка для разработки логики бота обусловлен его асинхронной природой и глубокой поддержкой Telegram Bot API версии 8.0+, включая функционал Telegram Stars и Mini Apps.¹⁰ Асинхронность критически важна для VPN-сервиса, так как операции создания ключей и взаимодействия с базой данных PostgreSQL не должны блокировать основной цикл обработки сообщений.

Структура асинхронных обработчиков и Middleware

Проект требует четкого разделения логики на уровни:

- **Уровень представления (Handlers):** обработка команд /start, управление кнопками Mini App и текстовыми сообщениями.¹²
- **Бизнес-логика (Services):** управление логикой выдачи пробных периодов, расчет реферальных бонусов и взаимодействие с API Marzban.¹³
- **Уровень данных (Repositories):** взаимодействие с PostgreSQL через SQLAlchemy 2.0 для хранения метаданных пользователей и истории транзакций.¹⁴
- **Middleware:** проверка статуса подписки пользователя перед доступом к функциям бота и логирование действий администратора (при соблюдении концепции Zero Logs для рядовых пользователей).

Концепция «Zero Logs» и безопасность данных

Обеспечение приватности в 2026 году требует полного отказа от хранения информации о сетевой активности пользователей. В рамках проекта реализуется стратегия тотального отключения журналов доступа в Xray core.¹⁵ Это достигается путем модификации объекта LogObject в конфигурационном файле Marzban.

Настройка параметров логирования в Xray для достижения «нулевого следа»:

- Установка loglevel в значение "none" исключает запись отладочной информации и ошибок.¹⁵
- Параметры access и error направляются в /dev/null или устанавливаются в "none", что предотвращает сохранение IP-адресов подключений и временных меток.¹⁵
- Отключение DNS-логирования (dnsLog: false) гарантирует, что история запросов к доменным именам не будет зафиксирована.¹⁵

Данная мера безопасности делает сервис устойчивым к юридическим запросам и техническому аудиту, так как данные, которые могли бы скомпрометировать пользователя, физически отсутствуют на сервере.

Финансовая экосистема: Telegram Stars и российские шлюзы

Монетизация сервиса в 2026 году опирается на гибридную модель оплаты, сочетающую внутреннюю валюту Telegram Stars (XTR) для глобальной аудитории и специализированные российские платежные решения для локального рынка.¹⁶

Интеграция Telegram Stars

Telegram Stars представляют собой виртуальную валюту для оплаты цифровых товаров и услуг внутри платформы. Процесс оплаты включает использование метода `sendInvoice` с указанием валюты XTR.¹⁶ Критически важным этапом является обработка `pre_checkout_query`, на который бот обязан ответить в течение 10 секунд, подтвердив наличие товара или готовность предоставить услугу.¹⁶

После успешного завершения транзакции бот получает объект `SuccessfulPayment`, содержащий `telegram_payment_charge_id`, который сохраняется в базе данных для возможности последующего возврата средств через метод `refundStarPayment` в случае возникновения споров.¹⁶

Российские платежные шлюзы: Prodamus и Robokassa

Для интеграции с Prodamus используется механизм генерации подписи HMAC-SHA256, обеспечивающий целостность данных при передаче между ботом и платежной формой.¹⁷ Алгоритм подписи требует строгого алфавитного порядка ключей в запросе, что гарантирует детерминированность хэша.

Математическая модель генерации подписи может быть представлена следующим образом:

Пусть $D = \{k_1 : v_1, k_2 : v_2, \dots, k_n : v_n\}$ — набор данных платежа.

Тогда строка для подписи S формируется как:

$$S = \text{join}(\text{sort}(\{k_i = v_i \mid v_i \neq \emptyset\}), "&")$$

Итоговая подпись H вычисляется с использованием секретного ключа K :

$$H = \text{HMAC-SHA256}(K, S)$$

.¹⁷

Robokassa предоставляет альтернативный канал оплаты, поддерживающий рекуррентные платежи и широкий спектр методов приема средств, включая СБП и банковские карты РФ. Использование асинхронных методов для генерации защищенных ссылок оплаты позволяет минимизировать время ожидания пользователя в Mini App.²⁰

Проектирование интерфейса Telegram Mini App

Mini App (ТМА) является основным инструментом взаимодействия пользователя с VPN-сервисом. Использование React в связке с Vite обеспечивает высокую скорость загрузки и плавность анимаций.¹⁴

Визуальный стиль и тактильный отклик

Стиль «Glassmorphism» достигается за счет использования CSS-свойств `backdrop-filter: blur()` и полупрозрачных фоновых цветов, что создает эффект матового стекла, гармонично вписывающегося в интерфейс Telegram.²²

Важной составляющей пользовательского опыта является Haptic Feedback (тактильная отдача). Через `@tma.js/sdk` реализуются различные типы вибраций:

- **Impact (light/medium)**: при переключении тумблера подключения VPN.²³
- **Notification (success)**: при успешной активации подписки.²⁴
- **Notification (error)**: при неудачной попытке оплаты или ошибке соединения.²⁵

Использование тактильной отдачи позволяет пользователю чувствовать физическую реакцию приложения на его действия, что значительно повышает уровень вовлеченности и удовлетворенности сервисом.

Инициализация проекта и структура репозитория

Для эффективной совместной работы Product Owner и разработчика (в лице ИИ-агента) необходимо внедрение строгой структуры проекта. Рекомендуется использование монорепозитория, где разделены зоны ответственности между ботом, веб-приложением и инфраструктурными скриптами.²²

Архитектура каталогов

- /
- └─ bot/ # Исходный код Telegram-бота (aiogram 3.x)
- | └─ handlers/ # Обработчики команд и событий
- | └─ services/ # Логика взаимодействия с Marzban API
- | └─ database/ # Модели SQLAlchemy и миграции Alembic
- | └─ middlewares/ # Фильтры и промежуточные слои
- └─ webapp/ # Frontend часть (React + Vite)
- | └─ src/ # Компоненты, хуки, стили (Tailwind)
- | └─ public/ # Статические ресурсы
- └─ infra/ # Инфраструктурные файлы
- | └─ docker-compose.yml
- | └─ nginx.conf # Конфигурация реверса-прокси
- | └─ xray_config/ # Шаблоны конфигураций для нод
- └─ CLAUDE.md # Правила разработки и стандарты кода
- └─ .env # Конфигурационные переменные (не для Git)

14

Регламент разработки: CLAUDE.md

Файл CLAUDE.md служит «инструкцией по эксплуатации» для ИИ-агента, определяя технологический стек, стандарты форматирования и ключевые команды проекта. Он гарантирует, что сгенерированный код будет соответствовать архитектурным принципам, заложенным на старте.¹⁴

Раздел	Содержание
Технологии	Python 3.11+, React 18, PostgreSQL, Docker
Стандарты	Type hinting, асинхронность, DRY, SOLID

Безопасность	Обязательная валидация hash в Mini App ²⁷
Команды	make build, make test, ruff check.

Настройка окружения и переменных

Файл .env должен содержать все необходимые ключи и параметры для интеграции компонентов системы. Особое внимание следует уделить разделению переменных для Marzban и Telegram.³

Переменная	Описание
TELEGRAM_API_TOKEN	Токен бота от BotFather
MARZBAN_API_URL	Адрес панели управления (например, http://localhost:8000)
MARZBAN_ADMIN_USERNAME	Логин администратора панели
MARZBAN_ADMIN_PASSWORD	Пароль администратора панели
POSTGRES_DSN	Строка подключения к БД
XTR_PROVIDER_TOKEN	Пустая строка для Stars или токен провайдера ¹⁶
PRODAMUS_SECRET	Ключ для проверки подписи уведомлений

Методология запуска: Роль администратора и задачи разработчика

Процесс инициализации проекта разделен на две параллельные ветки: ручные действия администратора (инфраструктура) и автоматизированная разработка через Claude Code (логика).

Обязанности Product Owner / Admin (Ручная настройка)

До начала активной фазы написания кода администратор должен выполнить следующие действия:

1. Подготовка VPS на Debian 12 и установка Docker.¹
2. Регистрация доменного имени и настройка A-записей для сервера.³
3. Получение SSL-сертификатов (например, через Let's Encrypt) для обеспечения работы HTTPS в Mini App.³
4. Регистрация бота в BotFather и создание необходимых команд меню.²⁷
5. Установка и базовая настройка панели Marzban через официальный скрипт установки.¹

Задачи для Claude Code (Инструкции для ИИ)

После передачи контекста через «Context Bridge» ИИ-агенту следует поручить:

- Генерацию базового скелета проекта согласно описанной структуре каталогов.
- Написание базового FastAPI-сервера для обработки вебхуков от Telegram и платежных систем.¹⁴
- Создание миграций базы данных для хранения информации о пользователях и их рефералах.
- Реализацию системы автоматической выдачи 24-часового триала через вызовы API Marzban `add_user`.⁹

Углубленная логика реферальной системы и триалов

Система бонусов «2+1» (за двух приглашенных — один месяц бесплатно) требует тщательного проектирования на уровне базы данных. В таблице Users необходимо хранить поле `referrer_id`, указывающее на пригласившего пользователя. При каждом успешном продлении подписки рефералом (статус `active` после оплаты), в таблице Referrals фиксируется событие. При достижении счетчика кратного двум, скрипт автоматически отправляет запрос к Marzban API для продления срока действия подписки пригласившего на 30 дней через метод `user_add_days`.⁹

Автоматическая выдача тестового периода на 24 часа реализуется через специальный обработчик команды `/start`. Бот проверяет наличие пользователя в БД; если запись отсутствует, вызывается метод API Marzban для создания временного аккаунта с параметром `data_limit` (например, 1 ГБ) и `expiration_date` равным текущему времени плюс 86400 секунд.⁵

Заключение и стратегические рекомендации

Реализация VPN-сервиса на стеке Marzban + aiogram 3.x + Telegram Mini App в 2026 году является наиболее технологически обоснованным решением для обхода современных систем цензуры. Использование протокола VLESS с транспортом Reality обеспечивает высокий уровень маскировки трафика, а интеграция с Telegram Mini App позволяет

создать максимально удобный пользовательский интерфейс, сопоставимый по качеству с нативными мобильными приложениями.

Ключевым фактором успеха проекта является строгое соблюдение концепции Zero Logs, что гарантирует безопасность пользователей и устойчивость бизнеса. Архитектура, основанная на асинхронном взаимодействии компонентов и масштабируемых нодах Marzban, позволяет сервису расти без потери производительности. Внедрение гибридной системы оплаты через Telegram Stars и локальные шлюзы минимизирует финансовые риски и расширяет потенциальную аудиторию проекта до глобальных масштабов.

Для старта разработки рекомендуется вставить предоставленный промпт в Claude Code и начать с генерации структуры папок, что заложит фундамент для последующей реализации бизнес-логики. Дальнейшие шаги должны включать настройку Reality-транспорта на подготовленном VPS и интеграцию платежных уведомлений для автоматизации процесса выдачи доступов.

Works cited

1. How to setup XRay VPN server on Linux with VMESS, VLESS, Trojan protocols - Euro-Space, accessed April 24, 2026, <https://www.euro-space.net/blog/virtual-server-for-vpn/tutorial/xray-vpn-setup-guide-vmess-vless-trojan-shadowsocks.php>
2. Xray Configuration for Marzban Deployment | PDF | Port (Computer Networking) - Scribd, accessed April 24, 2026, <https://www.scribd.com/document/985909624/Marzban-Master-Config>
3. strohsnow/Marzban-Guide - GitHub, accessed April 24, 2026, <https://github.com/strohsnow/Marzban-Guide>
4. Setting up VLESS in Marzban | AézaWiki - Aeza, accessed April 24, 2026, <https://wiki.aeza.net/en/guides/marzban/>
5. Introduction to Marzban | Marzban - GitHub Pages, accessed April 24, 2026, <https://gozargah.github.io/marzban/en/docs/introduction>
6. Gozargah/Marzban: Unified GUI Censorship Resistant Solution Powered by Xray - GitHub, accessed April 24, 2026, <https://github.com/gozargah/marzban>
7. mewhrzad/marzipan: A Python library that helps you easily use Marzban's API panel - GitHub, accessed April 24, 2026, <https://github.com/mewhrzad/marzipan>
8. strohsnow/Marzban-Node-Guide - GitHub, accessed April 24, 2026, <https://github.com/strohsnow/Marzban-Node-Guide>
9. aiomarzban - PyPI, accessed April 24, 2026, <https://pypi.org/project/aiomarzban/>
10. aiogram 3.19.0 documentation, accessed April 24, 2026, <https://docs.aiogram.dev/en/v3.19.0/>
11. aiogram · GitHub Topics, accessed April 24, 2026, <https://github.com/topics/aiogram>
12. 5 Tips for Building Chatbots with Python and Aiogram 3.x | by Ignaty Kashnitsky, accessed April 24, 2026, <https://python.plainenglish.io/5-tips-for-building-chatbots-with-python-and-aiogram->

[3-x-d34feeb18d2f](#)

13. Elegant Telegram Stars payments with aiogram 3.x — invoices, refunds, paid media, and balance tracking. - GitHub, accessed April 24, 2026, <https://github.com/bohd4nx/stars-payment>
14. Template for building scalable Telegram bots with aiogram + Mini App (WebApp) - GitHub, accessed April 24, 2026, <https://github.com/MrConsoleka/aiogram-miniapp-template>
15. 日志配置| Project X, accessed April 24, 2026, <https://xtls.github.io/config/log.html>
16. Bot Payments API for Digital Goods and Services - Telegram APIs, accessed April 24, 2026, <https://core.telegram.org/bots/payments-stars>
17. Документация для самостоятельной интеграции сервисов ..., accessed April 24, 2026, <https://help.prodamus.ru/payform/integracii/rest-api/instrukcii-dlya-samostoyatelna-ya-integracii-servisov>
18. SuccessfulPayment - aiogram 3.27.0 documentation, accessed April 24, 2026, https://docs.aiogram.dev/en/latest/api/types/successful_payment.html
19. StarTransaction - aiogram 3.22.0 documentation, accessed April 24, 2026, https://docs.aiogram.dev/en/v3.22.0/api/types/star_transaction.html
20. byBenPuls/robokassa: Unofficial Async Robokassa Python API | Star if you like it! - GitHub, accessed April 24, 2026, <https://github.com/byBenPuls/robokassa>
21. @telegram-apps/sdk-react examples - CodeSandbox, accessed April 24, 2026, <https://codesandbox.io/examples/package/@telegram-apps/sdk-react>
22. Telegram Mini App Template - CodeSandbox, accessed April 24, 2026, <https://codesandbox.io/p/github/Agunivers7/tg-mini-app-templat>
23. Haptic Feedback in React Native - Instamobile, accessed April 24, 2026, <https://instamobile.io/blog/haptic-feedback-react-native/>
24. Send Haptic Feedback - World Developer Docs - Worldcoin, accessed April 24, 2026, <https://docs.world.org/mini-apps/commands/send-haptic-feedback>
25. [Web] Beginning Telegram Mini Apps - Haptic Feedback | B4X Programming Forum, accessed April 24, 2026, <https://www.b4x.com/android/forum/threads/web-beginning-telegram-mini-apps-haptic-feedback.162253/>
26. How to Structure a Monorepo with Docker - OneUptime, accessed April 24, 2026, <https://oneuptime.com/blog/post/2026-02-08-how-to-structure-a-monorepo-with-docker/view>
27. Telegram Mini Apps Creation Handbook - DEV Community, accessed April 24, 2026, https://dev.to/simplr_sh/telegram-mini-apps-creation-handbook-58em
28. Telegram Bot | Marzban - GitHub Pages, accessed April 24, 2026, <https://gozargah.github.io/marzban/en/docs/telegram-bot>
29. Webhook - aiogram 3.27.0 documentation, accessed April 24, 2026, <https://docs.aiogram.dev/en/v3.27.0/dispatcher/webhook.html>